PRODUCT REQUIREMENTS DOCUMENT

Focus & Reminder

Android Productivity App | Enforceable App-Usage Limits & High-Attention Reminders

VERSION 1.1	STATUS Scope Frozen (Decisions Resolved)	PLATFORM Android 10+ (API 29–35)
PRODUCT CATEGORY Digital Wellbeing & Productivity	WORKING NAME Focus & Reminder (temporary)	TARGET RELEASE MVP (Local-First)

1. Executive summary

1.1 Product vision

Build an Android application that helps users regain control over their attention through enforceable app-usage limits and high-attention reminders.

The application combines two core capabilities:

1. **Focus enforcement:** Restrict selected applications when their usage exceeds a defined limit. Use a prominent blocking screen and a deliberately reflective emergency-access process instead of a simple dismiss button.
2. **Rich reminders:** Create reminders that are difficult to overlook, with prominent visual presentation, configurable sound and vibration, optional images, and explicit acknowledgement.

In later phases, the product will evolve into a personal productivity interface that integrates Google Calendar, displays tasks and events together, and provides a large home-screen widget for daily planning.

1.2 Product philosophy

The product should follow five principles:

- **Attention over notification:** Important reminders should demand deliberate attention rather than disappear into a notification list.
- **Intentional friction:** Access restrictions should encourage users to reconsider their actions before bypassing a limit.
- **Privacy by default:** Monitor only explicitly selected applications. Do not inspect browser tabs, URLs, browsing history, or page contents.
- **Local-first operation:** Core functionality should work without an internet connection or a backend service.
- **Progressive development:** Build and validate the essential mechanisms before introducing calendar synchronization, widgets, and other advanced features.

1.3 Target user

The initial target user is someone who:

- Repeatedly exceeds intended time limits on distracting apps.
- Wants meaningful restrictions rather than easily dismissed notifications.
- Misses reminders because of silent phones, notification overload, or lack of visual prominence.
- Wants a single interface for reminders and daily planning.
- Values privacy and prefers local device functionality.

The first release is intended primarily for individual users, not enterprise device management or parental control.

2. Problem statement

Problem 1: Uncontrolled app consumption

Users may intend to spend 10–15 minutes on an application but continue scrolling well beyond that duration.

Conventional screen-time notifications often rely on self-control at the moment of interruption. If the user can immediately dismiss the warning, the intervention may have little practical effect.

Product opportunity: Enforce a usage budget, interrupt the selected application, and require deliberate reflection before exceptional access.

Problem 2: Low-attention reminders

Conventional calendar reminders may be easy to overlook when the user is engaged in another application, misses the initial alert, or cannot hear the notification.

Product opportunity: Create a richer reminder experience that supports prominent visual alerts, configurable sound and vibration, optional images, and acknowledgement.

Problem 3: Fragmented daily planning

Users may create events in Google Calendar but need a better interface for viewing tasks, reminders, and upcoming events together.

Product opportunity: Gradually build a daily-planning interface that complements existing calendars instead of requiring users to migrate immediately.

---

3. Product goals and non-goals

3.1 Goals

1. Limit selected-app usage to no more than the configured budget within a rolling 60-minute window.
2. Interrupt the selected app when the budget is exhausted.
3. Enforce a minimum 30-minute cooldown after a blocking event.
4. Make bypassing the blocker a deliberate process involving reflective written answers.
5. Deliver reliable local reminders with prominent visual presentation where Android permits it.
6. Support configurable reminder content, scheduling, acknowledgement, and snoozing.
7. Keep core features operational offline.
8. Establish an architecture that can later support calendar integration and a daily-planner widget.

3.2 Non-goals for the MVP

The first release will not attempt to:

- Track browser tabs, URLs, browsing history, or page contents.
- Restrict websites or browser-based versions of selected applications.
- Synchronize Google Calendar in both directions.
- Provide cloud accounts, cross-device synchronization, or a mandatory backend.
- Provide social accountability, leaderboards, or gamification.
- Build an advanced analytics dashboard.
- Guarantee unrestricted full-screen takeover in every Android state.
- Guarantee tamper-proof enforcement against device restarts, permission revocation, force-stopping, or uninstalling.
- Replace every capability of Google Calendar.

- Deliver a complex daily-planner widget in the initial release.

---

4. Feature priority framework

Use the following priorities throughout development.

- **P0 — Essential:** Required for the first usable release.
- **P1 — Next phase:** Important, but not necessary to validate the core product.
- **P2 — Later:** Valuable extensions after the core application is reliable.

---

5. Functional requirements

5.1 Feature A: App selection and permissions

PRIORITY P0

Description

Allow users to choose which installed applications should be subject to usage restrictions.

Requirements

- The user must explicitly select applications to restrict.
- The user can add or remove selected applications.
- The interface should display each application's name and icon.
- The application must explain why each required Android permission is needed.
- Permission setup must distinguish usage monitoring from the mechanism used to display a blocking screen.
- If a required permission is unavailable or revoked, the app should show a clear status and recovery instructions.
- The app must not claim that restrictions are active when the required mechanism is not operational.
- Browser applications should not be monitored for their tab contents or URLs.

Acceptance criteria

- A user can select an installed application and configure a limit.
- The selected applications persist across ordinary app restarts.
- Permission loss is detected and communicated.
- The user can distinguish an active restriction from a configuration that is not currently enforceable.

5.2 Feature B: Rolling-hour usage budget

PRIORITY P0

Description

Track foreground usage of selected applications and enforce a maximum usage budget within a rolling 60-minute window.

Default configuration

Parameter	Default	Configurable
Usage budget	30 minutes	Yes (User-configurable, default 30 min)
Rolling window	60 minutes	Fixed at 60 minutes for MVP
Warning threshold	25 minutes of usage (or Budget - 5 min)	Calculated relative to budget
Minimum cooldown	30 minutes	Fixed at 30 minutes for MVP
Session grace period	5 minutes	Fixed at 5 minutes

The rolling usage budget is configurable per app or globally, defaulting to 30 minutes.

Definition of usage

Usage means the time a selected application is considered to be in the foreground and actively in use, based on Android signals available to the app.

The implementation should exclude periods where the selected application is known not to be in the foreground, including ordinary transitions to other applications and screen-off periods.

The exact tracking mechanism must be validated on real devices. Android usage-event data and permitted foreground-detection mechanisms have different reliability and background-execution constraints.

Rolling-window policy

At any point in time, calculate the accumulated foreground usage of the selected application during the preceding 60 minutes.

Let:

- B = 30 minutes be the usage budget.
- W = 60 minutes be the rolling window.
- $U(t)$ be tracked usage within the window ending at time (t).

The blocking condition is:

BLOCKING CONDITION

$$U(t) \geq B$$

The application should warn at the configured warning threshold, then block when the budget is exhausted.

Usage must age out of the rolling window according to its original timestamps. Starting a new session must not reset the budget.

Five-minute session grace period

The grace period defines whether two periods of app usage are considered part of the same session.

- If the user leaves the selected application and returns within five minutes, continue the existing session.
- If the user remains away for more than five minutes, begin a new session when they return.
- The session boundary does not reset the rolling usage budget.
- The grace period must not grant additional usage beyond the 30-minute budget.
- Screen-off periods and transitions between apps must not be counted as foreground usage.

Warning behaviour

When tracked usage reaches 25 minutes:

- Show a prominent warning when the platform permits.
- Clearly communicate the remaining permitted usage.
- Avoid requiring the user to navigate away from their current activity simply to understand the warning.
- If the warning cannot be presented at the exact threshold, show it at the next permitted opportunity and record the delay for testing.

Acceptance criteria

- Usage accumulates across multiple sessions within the same rolling window.
- A five-minute break preserves session continuity.
- A break longer than five minutes creates a new session without resetting the usage budget.
- Once the budget is exhausted, the application enters the blocked state.
- Usage outside the rolling window is no longer counted.
- Screen-off time does not count as foreground usage.

5.2.1 Feature B.2: Daily total app limit

PRIORITY P0

Description

Allow the user to configure an overall daily usage limit for each restricted application. The user manages their individual usage sessions across the day to stay within this cumulative daily allocation.

Requirements

- The user can configure a total daily limit per restricted application (e.g., 1 hour, 2 hours, or custom duration).
- The application tracks total foreground usage accumulated since 00:00:00 (midnight) in the device's local timezone.
- Daily usage accumulates across all sessions throughout the calendar day.
- When daily accumulated usage reaches the configured daily limit:
  - The application enters the blocked state for the remainder of the calendar day.
  - The blocking screen indicates that the daily total limit has been reached and shows the countdown to midnight reset.
- Daily usage resets automatically at midnight (00:00:00 local time).
- Both the rolling-hour limit and the daily total limit operate concurrently:
  - An application is blocked if *either* the rolling-hour budget is exhausted *or* the daily total limit is exhausted.
- Reflective emergency access (Feature D) remains available if exceptional access is required, following the full reflective question cycle.

Acceptance criteria

- Daily usage accurately tracks accumulated foreground time from midnight to midnight.
- When the daily limit is exhausted, the app is blocked even if the rolling 60-minute window has available budget.
- The blocker clearly informs the user that the daily limit was reached and shows time until midnight reset.
- At midnight, daily usage resets to zero and access is restored (provided any rolling-window cooldown is also satisfied).
- User can configure different daily limits for different restricted apps.

5.3 Feature C: Blocking screen and cooldown

PRIORITY P0

Description

When either the rolling usage budget or the daily total limit is exhausted, interrupt the selected application with a prominent blocking interface.

Required behaviour

- The blocker should prevent ordinary continued use of the selected application while the restriction is active.
- The blocker should explain why access is restricted (distinguishing between rolling-budget exhaustion and daily-limit exhaustion).
- It should display the remaining cooldown or time until midnight reset where practical.
- There must be no ordinary visible stop, dismiss, or disable button.
- The user should not be able to clear the cooldown merely by navigating to the home screen and reopening the selected application.
- The blocker should apply when the user returns through ordinary navigation or an app link, insofar as the chosen Android mechanism supports it.
- The cooldown must survive ordinary process death and app restarts.
- The blocker must not prevent access to unrelated applications or essential device functions.

Cooldown calculation

Access may be restored only when applicable restriction conditions are satisfied:

1. **For rolling-window exhaustion:**
   - The minimum 30-minute cooldown has expired:

     UNLOCK CONDITION
     $t_{\text{unlock}} \geq t_{\text{block}} + 30 \text{ minutes}$ - The selected application's rolling usage budget is available again under the rolling-window policy.
2. **For daily-limit exhaustion:**
   - The lock persists until midnight (00:00:00 local time), when the daily quota resets to zero.

This prevents the minimum cooldown and rolling-window rules from contradicting each other.

Restart policy

For the initial version, the user has explicitly chosen to allow a restart to reset the active restriction state.

- After a device restart, begin a fresh tracking period.
- Do not attempt to preserve the previous cooldown as an unbreakable restriction.
- Clearly document that restarting the device can bypass the intended enforcement.
- Do not represent this behaviour as tamper-proof.

This is an intentional MVP trade-off, not an accidental implementation detail.

Acceptance criteria

- A selected application is interrupted when the limit is exhausted.
- The blocker has no ordinary dismiss or stop control.
- Ordinary navigation does not reset the cooldown.

- Access remains blocked for at least 30 minutes after the blocking event.
- Permission failure or unsupported device behaviour is surfaced rather than silently treated as successful enforcement.
- Restart behaviour matches the documented MVP policy.

5.4 Feature D: Reflective emergency access

PRIORITY P0

Description

Provide a deliberate emergency-access path without exposing a conventional bypass button.

The objective is to introduce a moment of reflection before the user resumes potentially unproductive usage.

Interaction requirements

- The ordinary blocking screen must not expose a simple dismiss or disable button.
- Emergency access should require navigation through a separate, deliberate flow.
- The user must answer four or five open-ended text questions.
- All required answers must be provided before the temporary override is granted.
- The user sets their desired access duration (e.g. number of minutes) as part of completing the reflection flow.
- The questions should be visible, readable, and answerable using the normal keyboard.
- The flow operates completely offline without requiring internet access or an AI service.
- Answers are retained locally in the device database so the user can review and reflect on their history. Answers are never processed, analyzed, or transmitted externally.
- The override is strictly temporary and automatically re-locks the app once elapsed.

Initial question set

1. What specific task requires you to open this application right now?
2. Why does this task need to happen now rather than after your cooldown?
3. What concrete outcome do you expect from this session?
4. Could you accomplish the same goal later, after using the next 60 minutes productively?
5. What will you do immediately after completing the task, and how will you avoid falling back into endless scrolling?

Additionally, the user specifies the exact access duration needed to complete this task.

Override policy

- **User-Defined Window:** The user specifies their required access time during the 5-question submission.
- **Automatic Re-Lock:** Once the granted access window expires, the application is immediately locked again.
- **Repeat Cycle Requirement:** If the user requires more time after expiration, they cannot simply extend it; the full 5-question reflective cycle must be repeated in full.
- **Continuous Tracking:** Normal usage continues to accumulate during the override window.
- **Local Review:** Past reflections are saved in a local journal within the app for personal review.

Acceptance criteria

- The ordinary blocker does not display a bypass button.
- The override requires all 5 completed written responses and a specified access duration.
- Empty responses cannot satisfy a required question.
- The app locks immediately once the specified access time elapses.
- Extending or getting additional access requires repeating the entire 5-question cycle.

- Reflective answers are stored locally and privately for user reflection.
- The override flow operates completely offline.

The application should introduce friction without obstructing emergency calls or other essential device functionality.

5.5 Feature E: Local reminder creation and management

PRIORITY P0

Description

Provide a dedicated reminder system that works independently of Google Calendar.

Reminder fields

Each reminder should support:

- Title — required.
- Description or notes — optional.
- Scheduled date and time — required for a one-time reminder.
- Repeat schedule — optional.
- Optional image.
- Sound preference.
- Vibration preference.
- Acknowledgement policy.
- Snooze options.
- Reminder status.

Reminder actions

The user should be able to:

- Create a reminder.
- Edit a reminder.
- Reschedule a reminder.
- Delete a reminder.
- Mark a reminder as completed.
- Snooze a reminder.
- Configure repeating reminders.

Changes must update the local scheduled notification or alarm.

Local-first requirement

- Reminder creation must not require account registration.
- Reminder scheduling must not require internet access.
- Reminder definitions must be stored locally.
- Reminder delivery must not depend on a cloud server.
- Device restart handling must be implemented and tested.
- Where the operating system permits, scheduled reminders should be restored after reboot.
- The app must communicate relevant permission or scheduling limitations.

Acceptance criteria

- A reminder can be created and edited without network access.
- Changes to a reminder update its scheduled delivery.
- Snoozing schedules the next alert.
- Completed or deleted reminders do not continue firing unexpectedly.
- Reminder definitions remain available after ordinary app restarts.

5.6 Feature F: Prominent full-screen reminders

PRIORITY P0, SUBJECT TO PLATFORM FEASIBILITY

Description

At the scheduled time, present an important reminder in a dedicated, high-attention interface rather than relying only on a small notification.

Desired presentation

The reminder screen should support:

- Large, readable title.
- Description or notes.
- Optional image.
- Scheduled time.
- Sound and vibration indicators where useful.
- Acknowledge or complete action.
- Snooze action.
- Dismissal behaviour appropriate to the reminder's configured policy.

Behaviour requirements

- The app should attempt to display a prominent reminder over other apps at the scheduled time using platform-compliant mechanisms (`SYSTEM_ALERT_WINDOW` or full-screen intent notifications).
- **Selective Persistence Policy:**
  - **P0 Reminders (Essential):** Stick on-screen indefinitely until the user explicitly acknowledges, completes, or snoozes them.
  - **P1 / P2 Reminders (Standard):** Display prominently for a configured display duration, and then dismiss to a persistent system notification in the notification tray if unacknowledged.
- **Platform Fallback:** If full-screen presentation is not permitted or restricted by the OS, fallback immediately to a high-priority heads-up notification with sound/vibration and action buttons.
- If an alert is missed, the reminder remains available in the notification shade and in-app schedule.

Sound and vibration

- Sound must respect the user's selected preference and Android's notification-channel behaviour.
- If the phone is in silent mode, the reminder must remain silent.
- If the phone is in vibrate mode, vibration may be used according to the user's preference and system settings.
- The application must not override silent mode to force sound.
- Visual reminders should still be attempted when sound is unavailable.

Acceptance criteria

- A scheduled reminder triggers locally at the intended time, subject to documented Android scheduling constraints.
- The reminder is prominent and contains the configured content.
- Acknowledgement and snooze actions work correctly.
- Sound and vibration behaviour respects device settings.
- Failure to obtain a required permission is clearly communicated.
- The app does not falsely claim guaranteed full-screen takeover.

5.7 Feature G: Offline operation and privacy

PRIORITY P0

Requirements

The following features must work without an internet connection:

- App selection and configuration.
- Usage tracking, to the extent supported by local Android APIs.
- Rolling-window budget calculation.
- Blocking and cooldown logic.
- Reflective emergency access.
- Reminder creation and editing.
- Local reminder scheduling and delivery.
- Reminder acknowledgement and snoozing.

Privacy boundaries

- Do not inspect browser tabs.
- Do not collect URLs or browsing history.
- Do not capture screen contents.
- Do not transmit usage histories or reflective answers to a server.
- Do not require a cloud account for core functionality.
- Request only the Android permissions required for documented features.
- Explain permission use in clear language.

Acceptance criteria

- Core features work in airplane mode after setup.
- The app does not require an application backend.
- No network transmission is needed for ordinary usage enforcement or local reminders.
- Permission requests explain their purpose.
- Sensitive local information is not unnecessarily retained.

---

6. Later-phase features

These are part of the long-term product direction but are not required for the first release.

Phase 2: Google Calendar integration

PRIORITY P1

Initially read existing calendar events, subject to permission and account synchronization.

Requirements:

- Request calendar access only when the user enables the integration.
- Display existing events alongside locally created reminders.
- Distinguish calendar events from locally owned reminders.
- Handle missing or revoked permissions.
- Avoid duplicating events that are already represented locally.
- Make clear whether an event is editable from the app or read-only.

Phase 3: Calendar rescheduling and synchronization

PRIORITY P1

Allow users to reschedule calendar events through the app and propagate changes to the source calendar.

Requirements:

- Identify the source event and calendar.
- Update the source event where the API and permissions permit.
- Handle recurring events and individual occurrences.
- Handle concurrent changes made in Google Calendar.
- Avoid duplicate updates and notifications.
- Provide clear failure and retry states.
- Consider Google account authentication and the Google Calendar API if the Android Calendar Provider is insufficient.

Phase 4: Large daily-planner widget

PRIORITY P1

Create a large home-screen widget that displays the user's day.

Potential contents:

- Date and day progress.
- Upcoming events.
- Tasks and reminders.
- Completed and pending tasks.
- The next important deadline.
- Free intervals between scheduled events.
- Quick actions for creating tasks and reminders.

The widget must use the same underlying data model as the main app.

Phase 5: Enhanced productivity features

PRIORITY P2

Potential additions after validating the core product:

- Daily planning and prioritisation.
- More sophisticated reminder templates.
- Optional history of reflective overrides.

- Usage summaries and trends.
- More configurable usage budgets.
- Improved recurrence and scheduling controls.
- Optional backup and cross-device synchronization.
- Additional calendar integrations.

These features should be prioritised using user feedback rather than added speculatively.

---

7. Non-functional requirements

Reliability

- Persist reminder definitions and ordinary app configuration.
- Handle ordinary process termination.
- Recover scheduled reminders after device reboot where possible.
- Detect loss of essential permissions.
- Avoid silently dropping reminders.

Performance

- Avoid excessive polling.
- Minimise battery consumption.
- Keep the blocking decision logic efficient.
- Avoid unnecessary network requests.

Accessibility

- Support readable text sizes.
- Provide sufficient contrast.
- Label important controls for screen readers.
- Make the reflective questions accessible.
- Avoid relying solely on colour to communicate status.

Maintainability

- Separate product rules from Android-specific mechanisms.
- Keep rolling-window calculations testable without a device.
- Keep reminder scheduling separate from reminder presentation.
- Avoid embedding all business logic in UI components.
- Document significant architecture decisions.

Security

- Use platform-supported permissions.
- Do not attempt to disable essential system controls.
- Do not store sensitive answers unnecessarily.
- Make local data retention understandable.
- Do not claim tamper resistance that the application does not provide.

---

8. Success metrics

The following are provisional internal targets, not established performance results.

Metric	Initial target
Rolling-window calculation correctness	100% in deterministic unit tests
Correct cooldown calculation	100% in deterministic unit tests
Reminder scheduling reliability	At least 99% in controlled tests under supported conditions
Successful blocking events	At least 95% on the defined device test matrix
Emergency override correctness	100% of tested flows respect the configured access window
Offline core functionality	All required core flows pass airplane-mode testing
Setup completion	Establish a baseline, then improve through usability testing
Battery impact	Measure on representative devices before setting a numerical target

Measure both successful enforcement and platform limitations. Do not count an OS-restricted takeover as a successful full-screen alert.

---

9. Dependencies and risks

Android platform restrictions

App usage monitoring, overlay behaviour, foreground execution, exact alarms, and full-screen notifications are subject to Android permissions, OS versions, and distribution policies.

Device variation

Different manufacturers may restrict background activity or aggressively manage battery usage.

Restart bypass

The selected restart policy allows the restriction to reset. This is accepted for the initial version.

Calendar consistency

Calendar data may be unavailable, stale, read-only, or updated externally. Synchronization needs a separate design.

Reminder delivery

Exact delivery and full-screen takeover cannot be guaranteed in every system state. The product must provide compliant fallbacks and transparent status.

Battery usage

Continuous monitoring can impose battery costs if implemented inefficiently. The tracking approach must be validated before implementation is finalised.

---

10. Open decisions register (All Resolved)

The open decisions identified during requirements gathering have been resolved and frozen for MVP development:

#	Decision Item	Resolved Position	Rationale & Architectural Impact
1	**Rolling usage budget**	Configurable, default 30 minutes in rolling 60m	Allows flexibility while providing a proven, strict default. Stored in local settings per restricted app.
2	**Reminder on-screen duration**	Selective: P0 sticky; P1/P2 timed	P0 reminders stick until acknowledged; P1/P2 display for configured duration, then move to notification tray.
3	**Calendar integration scope**	Google Calendar only; Phase 2 (post-MVP)	Keeps MVP focused on local focus & rich reminders; calendar integration will be strictly Google Calendar in Phase 2.
4	**Reflective answers retention**	Retained locally for self-review	Saved in local SQLite/Room database; never processed or transmitted externally. Supports user personal growth.
5	**Emergency access window**	User-defined duration; full cycle repeat	User inputs needed minutes during the 5 questions; locks upon expiry. Subsequent overrides require repeating all 5 questions.
6	**Android support matrix**	Android 10 (API 29) to Android 15 (API 35)	Covers current latest (Android 15) and 5 versions back. `minSdk = 29`, `targetSdk = 35`. Covers >90% of active Android devices.
7	**Fallback presentation**	High-priority system notification	When `SYSTEM_ALERT_WINDOW` or full-screen intent is constrained by OS, trigger high-priority heads-up notification.
8	**Daily total app limit**	Configurable per app, resets at midnight	Adds cumulative daily cap in addition to the rolling 60-minute window; app blocks for the day once limit is hit.

11. Release acceptance criteria

The MVP is ready for an initial personal-use release when:

- The user can select apps to restrict and configure both rolling budgets and daily total limits.
- The rolling budget (default 30 min in 60 min) is implemented and tested with deterministic tests.
- The daily total app limit tracks usage from 00:00 to 23:59:59 and resets automatically at midnight.
- The five-minute session grace period behaves independently of the rolling budget.
- The app warns and blocks at the configured thresholds for both rolling and daily limits.
- The cooldown lasts at least 30 minutes for rolling-window locks, or until midnight for daily-limit locks.
- The reflective override requires all 5 written responses, lets the user set their access window, and automatically re-locks upon expiration.
- Extending access requires repeating the full 5-question reflection cycle.
- Reflective answers are stored locally and accessible for personal review.
- Users can create, edit, reschedule, snooze, acknowledge, and delete local reminders.
- P0 reminders stick on-screen until acknowledged; P1/P2 reminders dismiss gracefully to the notification shade.
- Fallback notifications display reliably when full-screen takeover is not permitted.
- Silent and vibrate modes are respected.
- Core functionality works offline in airplane mode.
- Required permissions and known limitations are communicated clearly.
- Automated tests cover the critical usage and reminder state transitions.
- The application has been tested on at least one physical Android device running Android 10+.

DEFINITION OF SUCCESS

A reliable, privacy-conscious Android application that meaningfully interrupts unwanted app usage and makes important reminders substantially harder to overlook, without requiring a cloud backend for its core functions.